Evidencias – Semana 4: Python + MySQL (CRUD en consola)

FoodieGo (continuación del Mini-ORM) – Programación y Base de Datos | Estudiantes: alisson pinoargotty y damarys zambrano | Fecha: 13/10/2026

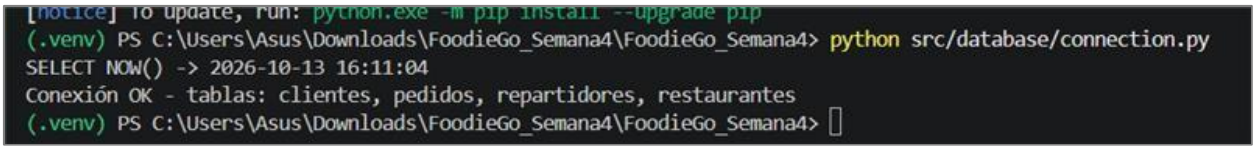
Módulo de consola en Python conectado a MySQL mediante variables de entorno (.env). Se continúa el Mini-ORM FoodieGo, migrándolo de SQLite a MySQL, con CRUD, consultas parametrizadas, validaciones de negocio y transacciones (commit en éxito, rollback ante error).

Equivalencias con el laboratorio

Laboratorio	FoodieGo
Categoría	Restaurante (entidad padre; conteo de pedidos por restaurante)
Producto	Pedido (FK a cliente, restaurante y repartidor; total >= 0)
SKU único	Email único (clientes) y placa única (repartidores)
Stock = 0	Total = 0

CP1 – Conexión y ejecución sin errores

Captura 1. Conexión a MySQL y tablas disponibles (python src/database/connection.py).



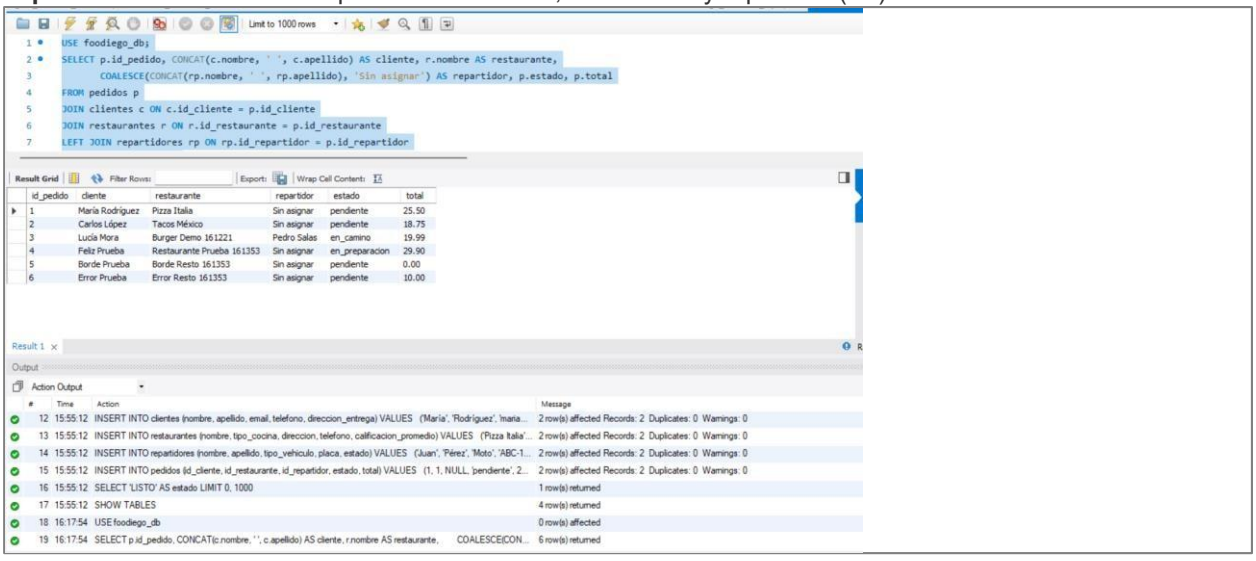
Captura 2. Demostración del CRUD en consola (python src/app_cli.py --demo).

```
(.venv) PS C:\Users\Asus\Downloads\FoodieGo_Semana4\FoodieGo_Semana4> python src/app_cli.py --demo
Conexión OK - tablas: clientes, pedidos, repartidores, restaurantes
--- Demo CRUD Consola FoodieGo ---
Registro guardado en restaurantes con ID 3
Registro guardado en clientes con ID 3
Registro guardado en repartidores con ID 3
Registro guardado en pedidos con ID 3
Registro 3 de pedidos actualizado
Pedido 3 asignado a Pedro
Pedidos (JOIN):
  (1, 'María Rodríguez', 'Pizza Italia', 'Sin asignar', 'pendiente', Decimal('25.50'))
  (2, 'Carlos López', 'Tacos México', 'Sin asignar', 'pendiente', Decimal('18.75'))
  (3, 'Lucía Mora', 'Burger Demo 161221', 'Pedro Salas', 'en_camino', Decimal('19.99'))
Pedidos por restaurante:
  ('Burger Demo 161221', 1)
  ('Pizza Italia', 1)
```

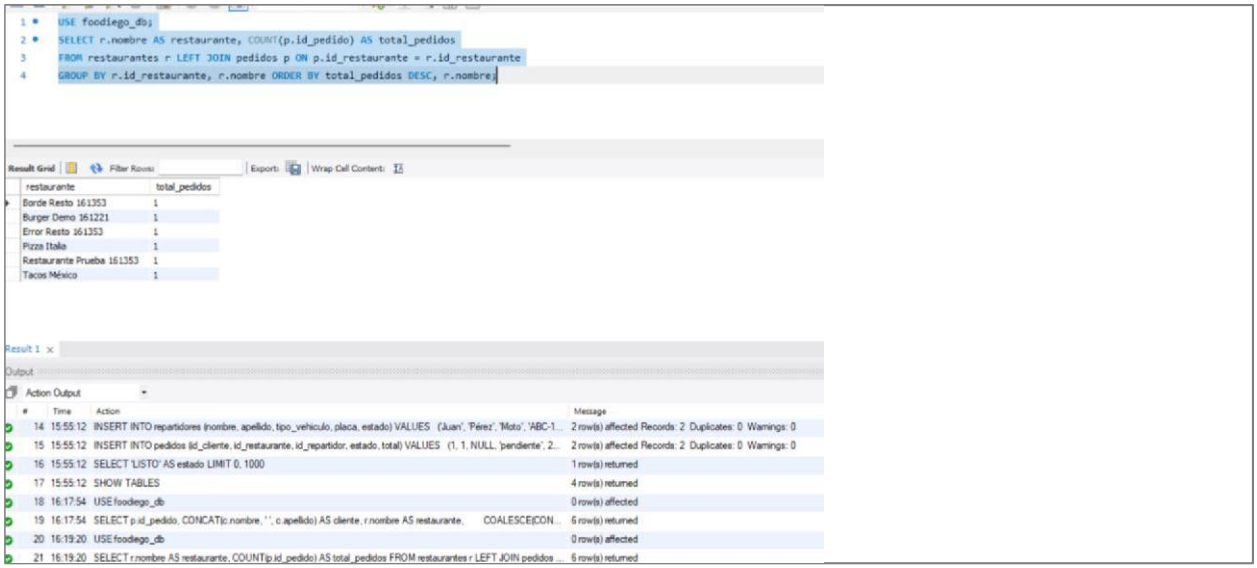
```
Conexión OK - tablas: clientes, pedidos, repartidores, restaurantes
--- Demo CRUD Consola FoodieGo ---
Registro guardado en restaurantes con ID 3
Registro guardado en clientes con ID 3
Registro guardado en repartidores con ID 3
Registro guardado en pedidos con ID 3
Registro 3 de pedidos actualizado
Pedido 3 asignado a Pedro
Pedidos (JOIN):
  (1, 'María Rodríguez', 'Pizza Italia', 'Sin asignar', 'pendiente', Decimal('25.50'))
  (2, 'Carlos López', 'Tacos México', 'Sin asignar', 'pendiente', Decimal('18.75'))
  (3, 'Lucía Mora', 'Burger Demo 161221', 'Pedro Salas', 'en_camino', Decimal('19.99'))
Pedidos por restaurante:
  ('Burger Demo 161221', 1)
  ('Pizza Italia', 1)
  ('Tacos México', 1)
```

CP2 – Operaciones reflejadas en MySQL

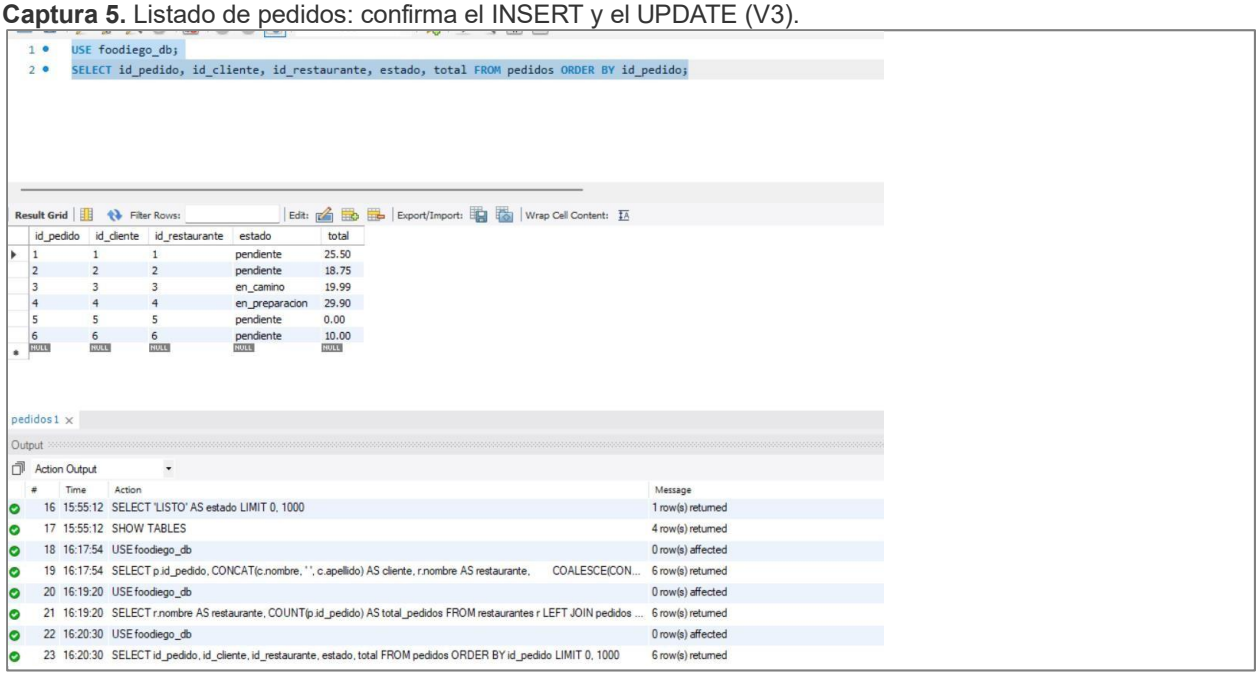
Captura 3. Consulta JOIN de pedidos con cliente, restaurante y repartidor (V1).



Captura 4. Conteo total de pedidos por restaurante (V2).



Captura 5. Listado de pedidos: confirma el INSERT y el UPDATE (V3).



CP3 – Pruebas mínimas (feliz, borde, error)

Caso	Qué se prueba	Resultado esperado	Pasa
Feliz	Crear restaurante, cliente y pedido válidos, actualizar total y estado, y listar con JOIN	Inserciones y actualización confirmadas en consola y en SELECT	
Borde	Pedido con total = 0, búsqueda vacía y búsqueda sin coincidencias	Total 0 aceptado; búsqueda vacía rechazada con mensaje; 0 resultados sin error	
Error	Email duplicado, cliente inexistente y borrado de un restaurante con pedidos	Excepción controlada, rollback y mensaje claro	

Captura 6 (parte 1). Ejecución de python src/pruebas_minimas.py: casos feliz y borde.

```
(.venv) PS C:\Users\Asus\Downloads\FoodieGo_Semana4\FoodieGo_Semana4> python src/pruebas_minimas.py
>>
Pruebas mínimas Semana 4 (FoodieGo) - 2026-10-13 16:13:53
== CASO FELIZ: crear, actualizar y listar con JOIN ==
Registro guardado en restaurantes con ID 4
Registro guardado en clientes con ID 4
Registro guardado en pedidos con ID 4
Registro 4 de pedidos actualizado
JOIN -> (4, 'Feliz Prueba', 'Restaurante Prueba 161353', 'Sin asignar', 'en_preparacion', Decimal('29.90'))
--> caso feliz: PASA

== CASO BORDE: total = 0 y búsqueda vacía ==
Registro guardado en restaurantes con ID 5
Registro guardado en clientes con ID 5
Registro guardado en pedidos con ID 5
Pedido con total 0 -> id=5, total=0.00
```

Captura 6 (parte 2). Caso error y resumen final.

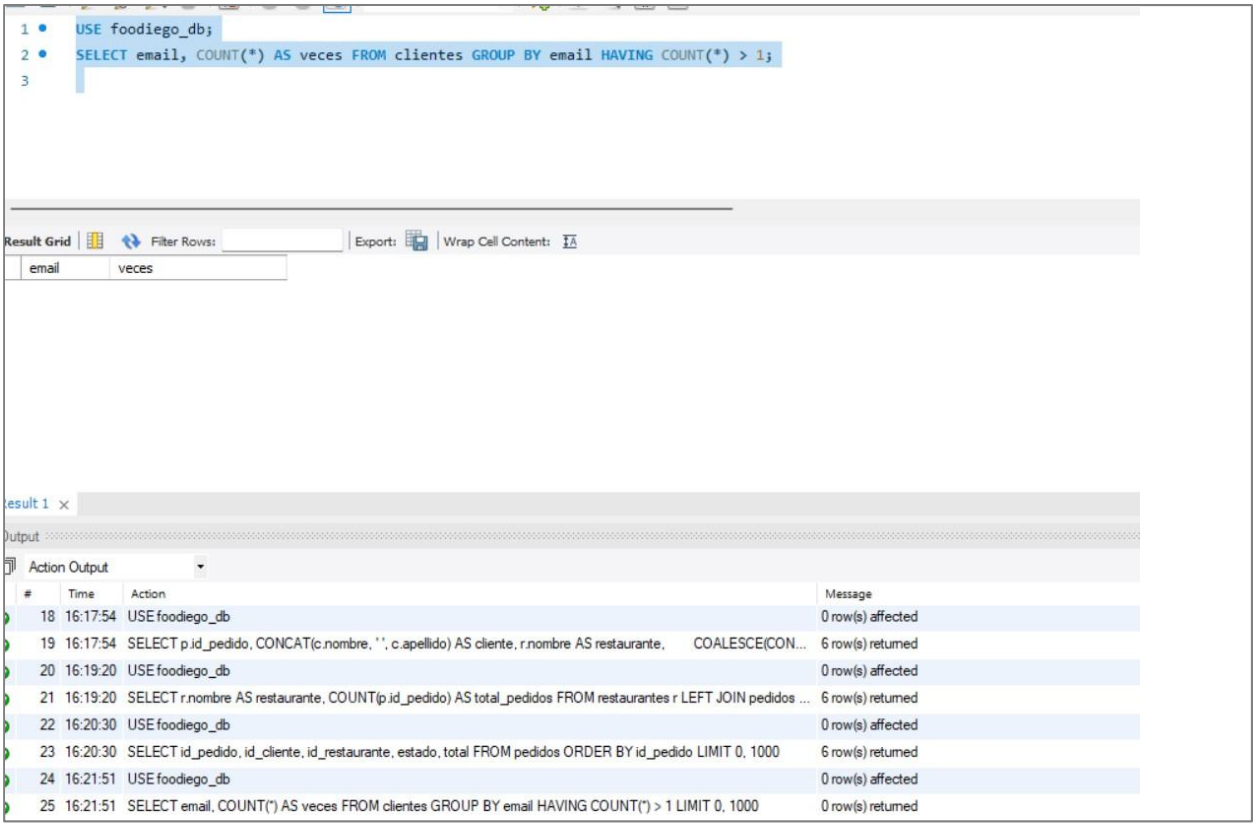
```
Pedido con total 0 -> id=5, total=0.00
Búsqueda vacía controlada: Escribe al menos un carácter para buscar
Búsqueda sin coincidencias -> 0 resultado(s)
--> caso borde: PASA

== CASO ERROR: email duplicado, FK inexistente y borrado con pedidos (rollback) ==
Registro guardado en restaurantes con ID 6
Registro guardado en clientes con ID 6
Registro guardado en pedidos con ID 6
Email duplicado controlado: No se pudo guardar en clientes: valor duplicado (email o placa ya existe)
Filas con ese email tras el rollback: 1 (esperado 1)
FK inexistente controlada: Cliente id=999999 no existe (FK inválida)
Pedidos antes: 6 / después: 6 (deben ser iguales)
Borrado bloqueado por FK: No se pudo eliminar de restaurantes: el registro tiene pedidos u otros datos relacionados (FK)
; no se puede eliminar
El restaurante sigue existiendo tras el rollback: True
```

```
== CASO ERROR: email duplicado, FK inexistente y borrado con pedidos (rollback) ==
Registro guardado en restaurantes con ID 6
Registro guardado en clientes con ID 6
Registro guardado en pedidos con ID 6
Email duplicado controlado: No se pudo guardar en clientes: valor duplicado (email o placa ya existe)
Filas con ese email tras el rollback: 1 (esperado 1)
FK inexistente controlada: Cliente id=999999 no existe (FK inválida)
Pedidos antes: 6 / después: 6 (deben ser iguales)
Borrado bloqueado por FK: No se pudo eliminar de restaurantes: el registro tiene pedidos u otros datos relaciona
; no se puede eliminar
El restaurante sigue existiendo tras el rollback: True
--> caso error: PASA

RESUMEN: feliz=PASA, borde=PASA, error=PASA
Guardado en C:\Users\Asus\Downloads\FoodieGo_Semana4\FoodieGo_Semana4\docs\salida_pruebas_20261013_161353.txt
(.venv) PS C:\Users\Asus\Downloads\FoodieGo_Semana4\FoodieGo_Semana4>
```

Captura 7. Verificación en MySQL: no hay emails duplicados (0 filas) tras el rollback (V4).



```
1 • USE foodiego_db;
2 • SELECT email, COUNT(*) AS veces FROM clientes GROUP BY email HAVING COUNT(*) > 1;
3
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

email	veces

esult 1 ×

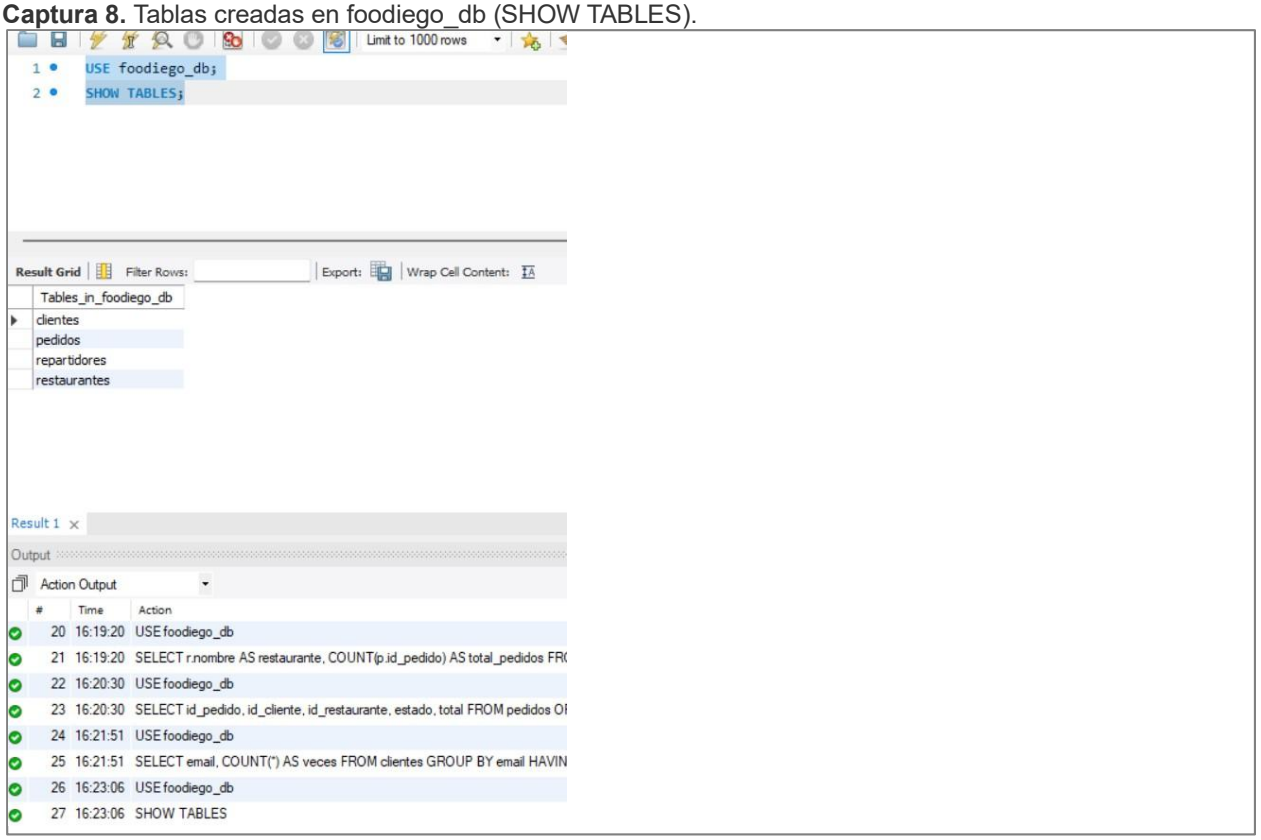
Output

Action Output

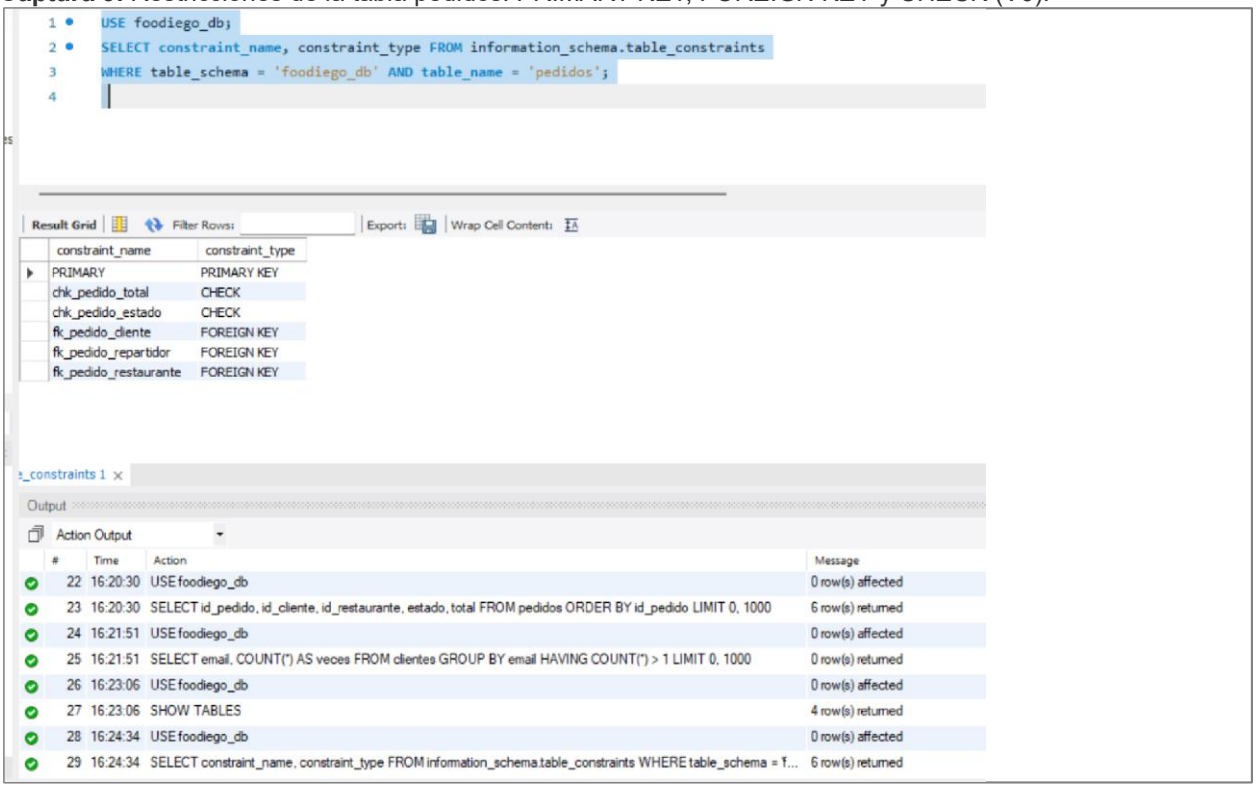
#	Time	Action	Message
18	16:17:54	USE foodiego_db	0 row(s) affected
19	16:17:54	SELECT p.id_pedido, CONCAT(c.nombre, ' ', c.apellido) AS cliente, r.nombre AS restaurante, COALESCE(CON...	6 row(s) returned
20	16:19:20	USE foodiego_db	0 row(s) affected
21	16:19:20	SELECT r.nombre AS restaurante, COUNT(p.id_pedido) AS total_pedidos FROM restaurantes r LEFT JOIN pedidos ...	6 row(s) returned
22	16:20:30	USE foodiego_db	0 row(s) affected
23	16:20:30	SELECT id_pedido, id_cliente, id_restaurante, estado, total FROM pedidos ORDER BY id_pedido LIMIT 0, 1000	6 row(s) returned
24	16:21:51	USE foodiego_db	0 row(s) affected
25	16:21:51	SELECT email, COUNT(*) AS veces FROM clientes GROUP BY email HAVING COUNT(*) > 1 LIMIT 0, 1000	0 row(s) returned

Cambios MySQL verificados

Captura 8. Tablas creadas en foodiego_db (SHOW TABLES).



Captura 9. Restricciones de la tabla pedidos: PRIMARY KEY, FOREIGN KEY y CHECK (V6).



Checklist Conductor / Navegador

Verificación	Navegador
CP1: conexión establecida y script sin errores	

CP2: INSERT/UPDATE/SELECT reflejados en MySQL	
CP3: 3 pruebas mínimas pasan y están documentadas	
.env fuera del repositorio; .env.example incluido	
requirements.txt actualizado; estructura /src /db /docs	

Reflexión y conclusión

Durante el laboratorio nos apareció el error 1044 (Access denied) al conectar con un usuario sin permisos sobre la base. Lo corregimos recreando la base y las tablas con un script limpio y configurando el .env con un usuario que tiene permisos. También resolvimos un ModuleNotFoundError instalando mysql-connector-python y ejecutando el programa desde la carpeta correcta del proyecto. Con esto comprobamos que las credenciales y los permisos se deben verificar antes de probar el código, y que las validaciones, las transacciones con rollback y las consultas parametrizadas mantienen consistente la información de FoodieGo en MySQL.